

Rapport de projet de fin d'études

Pour l'obtention du Diplôme de Licence en Informatique

Domaine : Mathématique & Informatique

Option : Informatique

Réalisés par :

- **NEGADI Mohammed Aymen**
- **BENDEHINA Malek Fatima Zohra**

Séssion 2 2026

THEME

Application web pour la gestion des actifs (logistiques) et inventaire

Universitaire

Jury

Examineur : Mme BENABDELLAH S.

Code Licence : INFO_6_063/2026

Remerciements

Remerciements de Bendehina Malek Fatima Zohra :

Tout d'abord, nous remercions Allah de nous avoir donné la santé, la volonté et la patience nécessaires pour accomplir ce travail.

Nous dédions ce travail à nos familles, qui ont été notre premier soutien tout au long de ce parcours. Leur amour, leurs sacrifices et leurs encouragements au quotidien nous ont donné la force d'aller jusqu'au bout. Aucun mot ne pourrait suffisamment exprimer notre reconnaissance envers eux.

Nous remercions également l'ensemble des enseignants du département informatique qui ont contribué à notre formation. Grâce à leurs cours, leurs conseils et leur dévouement, nous avons acquis les connaissances qui nous ont permis de réaliser ce projet.

Nous tenons aussi à remercier nos collègues de promotion pour ces années passées ensemble, pour les fous rires, les nuits de révision, les moments de galère partagés et l'entraide qui n'a jamais fait défaut. Cette promotion restera une page importante de notre vie.

À toutes ces personnes, nous disons simplement : merci.

Remerciements de Negadi Mohamed Aymen :

Tout d'abord, je remercie Allah le Tout-Puissant de m'avoir accordé la santé, la force, la patience et la réussite nécessaires pour mener à bien ce travail. El Hamdoulilah pour toutes Ses bénédictions.

Je tiens à remercier mon père, la personne qui m'a encouragé tout au long de ma vie et qui rêvait de me voir réussir. Aujourd'hui, même s'il n'est plus parmi nous et qu'il est auprès d'Allah, qu'Allah lui fasse miséricorde, je lui dédie cette réussite. J'ai poursuivi le chemin qu'il a commencé à tracer pour moi et j'ai accompli une étape qu'il souhaitait tant me voir accomplir. Que son âme repose en paix et qu'Allah l'accueille dans Son vaste Paradis.

J'adresse également mes plus sincères remerciements à ma famille pour son soutien indéfectible, ses encouragements, sa confiance et ses sacrifices tout au long de mon parcours académique.

Je tiens aussi à exprimer ma profonde gratitude à tous nos enseignants et professeurs qui nous ont formés, transmis leur savoir et accompagnés durant nos années d'études.

Enfin, je remercie toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail.

À tous, j'adresse mes plus sincères remerciements.

Table des matières

<i>Résumé.....</i>	<i>- 3 -</i>
<i>Abstract.....</i>	<i>- 4 -</i>
<i>Introduction Générale.....</i>	<i>- 5 -</i>
<i>Analyse des besoins</i>	<i>-7-</i>
<i>3.1 Identification des acteurs.....</i>	<i>- 8 -</i>
<i>3.2 Besoins fonctionnels par acteur</i>	<i>- 10 -</i>
<i>3.3 Modélisation statique – Diagramme de classes</i>	<i>- 12 -</i>
<i>Implémentation.....</i>	<i>-16-</i>
<i>4.1 Choix technologiques</i>	<i>- 17 -</i>
<i>4.2 Base de données.....</i>	<i>- 22 -</i>
<i>4.3 Fonctionnalités implémentées.....</i>	<i>- 25 -</i>
<i>Conclusion et perspectives</i>	<i>-30-</i>
<i>Références et Bibliographie</i>	<i>-33-</i>

Résumé

La gestion manuelle des équipements au sein des laboratoires et des services universitaires génère de nombreuses inefficacités : pertes de temps, erreurs récurrentes dans la tenue de l'inventaire et suivi insuffisant du matériel.

Ce projet a pour objectif la conception et le développement d'une application web centralisée et sécurisée dédiée à la gestion des actifs logistiques universitaires. La solution développée permet à cinq catégories d'acteurs : administrateur, responsable logistique, magasinier, responsable de laboratoire et professeur de consulter l'inventaire en temps réel, de tracer les flux de stocks (mouvements d'entrée et de sortie), et de déclarer instantanément les incidents sur le terrain à travers un système dynamique de signalements. L'accès aux différentes fonctionnalités est strictement protégé par un mécanisme de contrôle d'accès basé sur les rôles (RBAC) couplé à une authentification par jetons **JWT**.

Sur le plan technique, l'application adopte une architecture découplée. Le backend repose sur **Node.js** et **Express.js**, où l'accès et la persistance des données au sein du serveur **PostgreSQL** sont entièrement orchestrés par l'ORM **Prisma**, offrant une structure robuste à sept tables interconnectées et supervisée via **Prisma Studio**. L'interface utilisateur, fluide et responsive, est développée en **React.js** (avec Vite) et stylisée avec **Tailwind CSS**. Le développement a été réalisé sous Visual Studio Code, avec Git et GitHub pour la gestion des versions et la traçabilité du code source.

Ce système permet d'automatiser et de fiabiliser la gestion des équipements, réduisant ainsi les risques d'erreur et améliorant l'efficacité opérationnelle globale. Il s'inscrit dans une démarche plus large de numérisation et de modernisation des services administratifs et pédagogiques de l'établissement.

Mots-clés : application web, gestion des actifs logistiques, PostgreSQL, ORM Prisma, contrôle d'accès (RBAC), signalement d'incidents, automatisation, numérisation universitaire.

Abstract

The manual management of equipment across university laboratories and administrative departments gives rise to significant operational inefficiencies, including time loss, recurring inventory errors, and inadequate asset tracking.

This project presents the design and development of a centralized and secure web-based application for the management of logistical assets within a university setting. The system supports five categories of users : administrator, logistics manager, storekeeper, laboratory supervisor, and instructor enabling them to consult a real-time inventory, monitor equipment inflows and outflows (movements), and dynamically report field incidents through a streamlined signalment system. To ensure data integrity, access to functionalities is strictly protected by a Role-Based Access Control (RBAC) mechanism coupled with **JWT**-based authentication.

From a technical standpoint, the application implements a decoupled architecture. The backend layer is built on **Node.js** and **Express.js**, where data persistence and access within the **PostgreSQL** database are fully orchestrated by the **Prisma ORM**, creating a robust structure of seven interconnected tables monitored via **Prisma Studio**. The client-side interface is a fluid, responsive single-page application developed using **React.js** (with Vite) and styled with **Tailwind CSS**. Development was carried out using Visual Studio Code, with Git and GitHub providing version control and source code management.

The proposed system centralizes, secures, and automates equipment management, thereby minimizing human error and enhancing overall operational efficiency. This work contributes to a broader institutional initiative aimed at digitalizing and modernizing university administrative and academic services.

Keywords: web application, logistical asset management, PostgreSQL, Prisma ORM, Role-Based Access Control (RBAC), incident reporting, automation, university services digitalization.

Introduction Générale

La gestion des actifs logistiques désigne l'ensemble des processus permettant de recenser, de localiser, de suivre et d'entretenir les équipements d'une organisation tout au long de leur cycle de vie. Dans le contexte universitaire, ces équipements sont à la fois nombreux et hétérogènes : postes informatiques, mobilier pédagogique, instruments de mesure, matériel de laboratoire, entre autres. Leur suivi rigoureux constitue une nécessité opérationnelle, dans la mesure où ils représentent des ressources indispensables au bon fonctionnement des activités d'enseignement, de recherche et d'administration.

L'enjeu de cette gestion se manifeste à plusieurs niveaux complémentaires. Sur le plan pédagogique, la qualité des travaux pratiques est directement conditionnée par la disponibilité et le bon état fonctionnel du matériel. Sur le plan scientifique, les activités de recherche requièrent des instruments performants, correctement entretenus et disponibles en temps voulu. Sur le plan financier, les équipements constituent un investissement significatif pour l'établissement, ce qui rend indispensable l'optimisation de leur utilisation et de leur durée de vie.

À titre indicatif, un établissement universitaire de taille moyenne peut être amené à gérer plusieurs milliers d'équipements répartis sur l'ensemble de ses sites. Chaque année, un nombre important d'opérations (acquisitions, réparations, transferts, mises en réforme) génère un volume considérable d'informations qu'il convient d'enregistrer, de mettre à jour et de rendre accessibles aux acteurs concernés.

Or, malgré l'ampleur et la complexité de cette mission, les méthodes de gestion employées demeurent souvent rudimentaires. De nombreux services et laboratoires recourent encore à des registres papier ou à des tableurs de type Excel pour tenir leur inventaire. Si ces outils présentent l'avantage d'une mise en œuvre immédiate, ils révèlent rapidement leurs limites face au volume croissant des équipements et à la diversité des opérations à gérer. Les données sont fréquemment dispersées entre plusieurs fichiers non synchronisés, rarement mises à jour en temps réel, et difficiles à partager entre les différents intervenants.

Cette situation engendre des dysfonctionnements concrets : le responsable logistique ne dispose pas d'une vision fiable et actualisée du parc matériel, le responsable de laboratoire peine à localiser les équipements dont il a besoin, et le magasinier effectue des opérations de saisie redondantes et fastidieuses. Il en résulte des pertes de temps, des risques d'erreur et une prise de décision insuffisamment éclairée.

Ce constat soulève une question centrale, à laquelle ce projet entend apporter une réponse concrète : **comment garantir un suivi fiable et en temps réel des équipements universitaires, en substituant aux méthodes manuelles actuelles une solution numérique adaptée aux besoins des différents acteurs ?**

Pour répondre à cette problématique, le présent projet propose la conception et le développement d'une application web dédiée à la gestion des actifs logistiques d'un établissement universitaire. Cette solution vise à centraliser l'ensemble des informations relatives aux équipements au sein d'un système unique, accessible en temps réel par les acteurs habilités, selon leurs rôles et responsabilités respectifs.

Les bénéfices attendus sont multiples. Le responsable logistique disposera d'une vue globale et actualisée de l'inventaire, lui permettant de prendre des décisions éclairées en matière d'acquisition, de maintenance ou de réforme. Le magasinier bénéficiera d'outils de saisie et de mise à jour simplifiés, réduisant les risques d'erreur lors de l'enregistrement des équipements et de la gestion des mouvements. Le responsable de laboratoire pourra consulter en temps réel l'état et la disponibilité des équipements qui lui sont affectés, et formuler des demandes de manière structurée et traçable. Le professeur, quant à lui, pourra signaler tout dysfonctionnement et soumettre des demandes ponctuelles directement depuis l'interface de l'application.

Au-delà des bénéfices propres à chaque acteur, cette solution vise à renforcer la fiabilité des données, à réduire la charge des tâches administratives manuelles et à optimiser l'utilisation des ressources matérielles de l'établissement. Elle s'inscrit ainsi dans une démarche plus large de numérisation des services universitaires.

Analyse des besoins

L'analyse des besoins constitue une étape fondamentale dans le processus de conception d'un système informatique. Elle permet de définir avec précision les attentes des utilisateurs et de les traduire en fonctionnalités applicatives. Dans le cadre de ce projet, cette analyse porte sur l'identification des acteurs du système, la description de leurs besoins fonctionnels, et la modélisation statique des données à travers un diagramme de classes réajusté à la structure réelle de notre base de données.

3.1 Identification des acteurs

Un acteur est toute entité externe qui interagit avec le système afin d'accomplir un objectif précis. Son identification est indispensable pour délimiter le périmètre fonctionnel de l'application et définir les droits et responsabilités de chaque utilisateur. Le système implique cinq catégories d'acteurs, chacun disposant d'un rôle spécifique.

3.1.1 L'administrateur

L'administrateur est le gestionnaire principal de la plateforme. Il est responsable de la création, de la modification et de la suppression des comptes de tous les utilisateurs, de la configuration des laboratoires, et assure la supervision globale du système.

3.1.2 Le responsable logistique

Le responsable logistique assure le pilotage stratégique du parc matériel. Il dispose d'une vision globale sur l'état, la localisation et l'affectation des équipements, valide les demandes d'achat ou de réforme, suit les signalements d'incidents et analyse l'état du parc via un tableau de bord statistique.

3.1.3 Le magasinier

Le magasinier est le responsable opérationnel de la gestion physique des équipements. Il réceptionne le matériel depuis les demandes validées, gère les mouvements de stocks (entrées, sorties, transferts), traite les demandes d'emprunt physiques et se charge de l'impression des étiquettes d'identification unique à coller sur le matériel.

3.1.4 Le responsable de laboratoire

Le responsable de laboratoire consulte les équipements affectés à son entité, vérifie leur disponibilité et leur état de fonctionnement réel, traite les demandes intermédiaires des professeurs et soumet des demandes d'équipements (achat, remplacement, réforme) auprès du responsable logistique..

3.1.5 Le professeur

Le professeur interagit avec le système dans le cadre de ses activités pédagogiques. Il consulte le matériel disponible dans ses salles affectées, formule des demandes d'emprunt de matériel mobile ou des requêtes pour son laboratoire, et déclare instantanément tout incident constaté sur un équipement.

Le tableau ci-après récapitule les cinq acteurs et leur positionnement au sein du système.

Acteur	Rôle principal	Interactions avec le système
Administrateur	Gestion système et des comptes	Crée, modifie, supprime les accès de tous les acteurs et réinitialise les mots de passe.
Responsable logistique	Pilotage stratégique de l'inventaire	Consulte l'inventaire complet, pilote le dashboard, valide les demandes et traite les signalements.
Magasinier	Gestion physique et opérationnelle	Réceptionne les commandes validées, enregistre les flux de stock (mouvement d'entrée auto), gère les emprunts et imprime les étiquettes.
Resp. de laboratoire	Supervision locale du parc affecté	Supervise la disponibilité et l'état de son labo, valide les demandes des professeurs et soumet ses propres requêtes.
Professeur	Utilisation pédagogique des actifs	Consulte le matériel local, formule des demandes d'emprunt ou de laboratoire et déclare les signalements d'incidents.

Tableau 3.1 – Récapitulatif des acteurs du système

3.2 Besoins fonctionnels par acteur

Les besoins fonctionnels décrivent l'ensemble des actions que le système doit permettre à chaque acteur de réaliser. Ils sont présentés de manière structurée selon les privilèges d'accès de chaque profil.

3.2.1 L'administrateur

- S'authentifier de façon sécurisée au système.
- Créer de nouveaux comptes utilisateurs en associant un identifiant, un nom complet et un mot de passe initial à un rôle strict.
- Modifier les informations ou supprimer un compte utilisateur existant.
- Réinitialiser le mot de passe d'un utilisateur en cas de perte d'accès.
- Configurer les espaces physiques en créant des laboratoires et en leur attribuant un responsable.

3.2.2 Le responsable logistique

- S'authentifier au système.
- Consulter un tableau de bord (dashboard) analytique interactif alimenté par les statistiques réelles du parc.
- Consulter l'inventaire global des équipements avec un filtrage multicritères (par référence, état ou localisation).
- Valider ou rejeter les demandes d'achat, de remplacement, de réforme ou d'emprunt émises par les équipes.
- Traiter, suivre et gérer les signalements de pannes remontés par le terrain pour planifier les actions de maintenance.

3.2.3 Le magasinier

- S'authentifier au système.
- Réceptionner et enregistrer les nouveaux équipements directement depuis la liste des demandes préalablement validées par la logistique.
- Déclencher automatiquement la création d'une fiche équipement et d'un mouvement d'ENTREE lors de la validation d'une réception.
- Enregistrer manuellement les flux de stocks physiques (TRANSFERT, MAINTENANCE, SORTIE).
- Traiter et enregistrer les remises et retours physiques pour les demandes d'EMPRUNT des professeurs.
- Générer et imprimer les étiquettes d'identification physique dotées d'un identifiant unique pour chaque équipement.

3.2.4 Le responsable de laboratoire

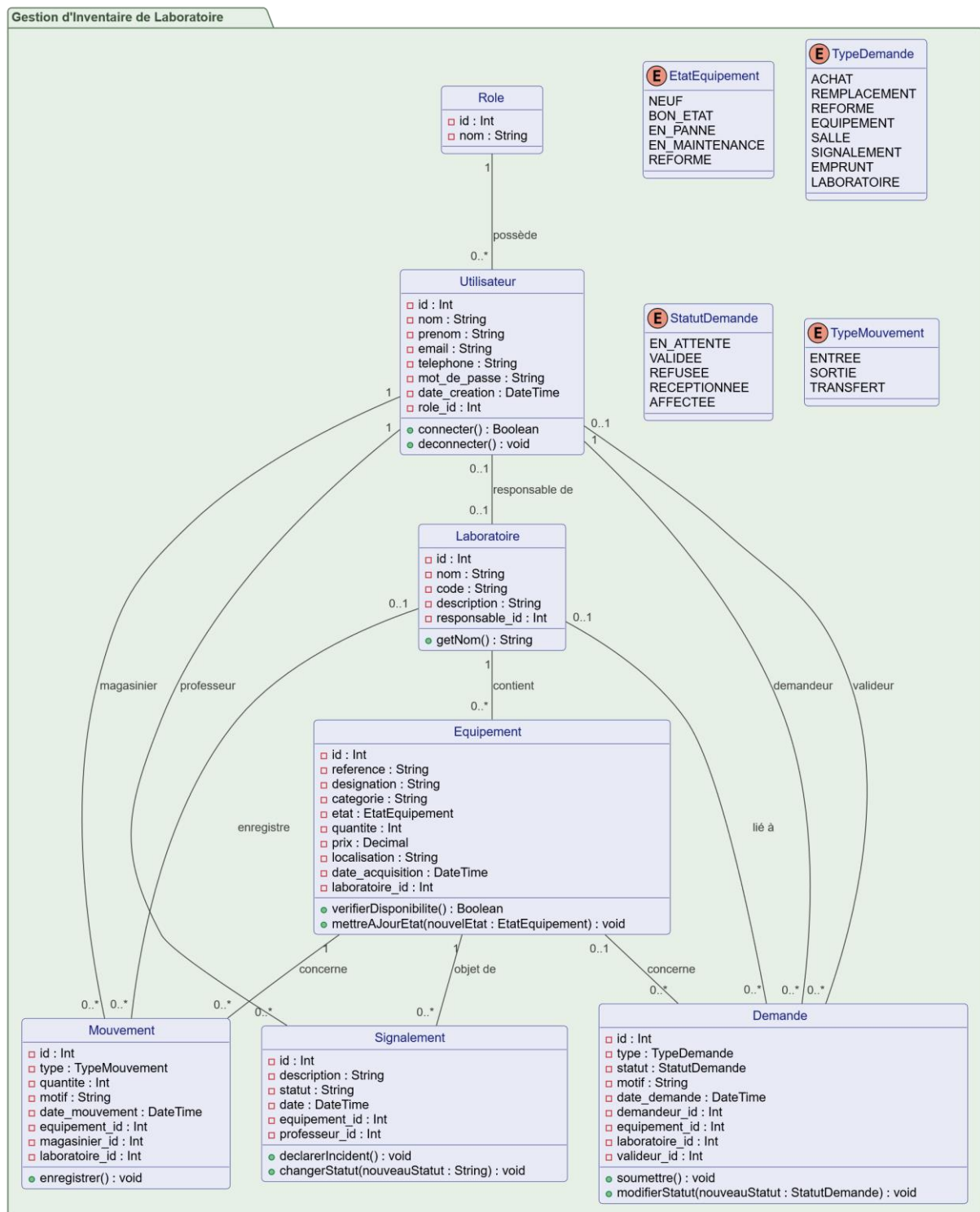
- S'authentifier au système.
- Superviser exclusivement les équipements affectés à son laboratoire en vérifiant en temps réel leur Disponibilité & État.
- Traiter et valider les demandes de matériel pédagogique ponctuel soumises par les professeurs pour leurs séances dans ses salles.
- Formuler des demandes de type LABORATOIRE (achat, remplacement ou réforme de matériel) auprès du responsable logistique.
- Consulter l'historique complet et le statut d'avancement de ses requêtes.

3.2.5 Le professeur

- S'authentifier au système.
- Consulter la liste et l'état du matériel disponible dans la salle ou le laboratoire qui lui est attribué.
- Déclarer instantanément un **Signalement** d'incident (bouton dédié ouvrant un modal de description) lorsqu'un équipement est défectueux.
- Formuler une demande d'**EMPRUNT** d'équipement mobile auprès du magasin central pour une durée déterminée.
- Formuler une demande de type **LABORATOIRE** adressée à son responsable pour l'aménagement ou l'amélioration de sa salle d'enseignement.
- Consulter l'historique de ses déclarations, signalements et demandes.

3.3 Modélisation statique – Diagramme de classes

La modélisation statique du système est représentée par le diagramme de classes ci-dessous. Il décrit la structure des données de l'application, les entités principales qui la composent, leurs attributs, leurs méthodes et les relations qui les unissent.



(Figure 3.1 – Diagramme de classes du système)

Le diagramme fait apparaître **sept classes principales** et s'appuie sur l'usage de types énumérés stricts (*Enums*) :

- **Utilisateur** : Classe centrale du système d'authentification. Elle stocke les informations de connexion de l'ensemble du personnel (identifiant unique, nom complet, nom d'utilisateur et mot de passe hashé via bcrypt). Pour des raisons de flexibilité et de sécurité (contrôle d'accès RBAC), elle possède une relation directe vers la table **Role**. Elle hérite de la classe abstraite **Personne**, qui encapsule les informations d'identité de base (identifiant, nom complet).
- **Role** : Cette table séparée est exclusivement dédiée à la gestion granulaire des privilèges logitiels. Elle définit les profils stricts de l'application (*Administrateur, Responsable Logistique, Magasinier, Responsable de Laboratoire, Professeur*) permettant de sécuriser l'accès aux interfaces et aux routes de l'API.
- **Laboratoire** : Représente les espaces physiques et les salles de l'établissement universitaire. Chaque laboratoire est identifié par un nom, un code interne unique, l'identifiant de son responsable et une capacité maximale. Il est lié à la classe **Équipement** par une relation d'agrégation/composition : un laboratoire contient une collection d'équipements.
- **Équipement** : Classe centrale du domaine métier. Elle stocke toutes les informations relatives aux actifs : nom, référence constructeur unique, catégorie, quantité, localisation et son état fonctionnel (géré par l'Enum *EtatEquipement* : NEUF, BON, MOYEN, PANNE, REFORME). Ses méthodes assurent le suivi rigoureux du cycle de vie des matériels.
- **Demande** : Recense toutes les requêtes formulées par les professeurs ou les responsables de laboratoire. Elle précise le type de demande (via l'Enum *TypeDemande* : ACHAT, REMPLACEMENT, REFORME, EMPRUNT, LABORATOIRE), la justification textuelle, la date, l'ID du demandeur et son statut évolutif (via l'Enum *StatutDemande* : EN_ATTENTE, VALIDEE, REFUSEE).
- **Mouvement** : Assure l'historique et la traçabilité physique complète de chaque flux de matériel. Elle enregistre le type d'opération (via l'Enum *TypeMouvement* : ENTREE, SORTIE, TRANSFERT, MAINTENANCE), la quantité concernée, les laboratoires source et destination, l'horodatage précis ainsi que l'utilisateur responsable de l'action.

- **Signalement** (*qui remplace l'ancienne entité Panne*) : Enregistre de manière exhaustive les incidents et dysfonctionnements matériels déclarés en temps réel sur le terrain. Elle associe l'équipement défectueux à l'utilisateur déclarant, intègre une description textuelle du problème et suit un cycle de statut évolutif (*Signalé, En cours de réparation, Résolu*).

En synthèse, l'analyse menée dans ce chapitre a permis d'identifier les cinq acteurs du système et de définir avec précision les fonctionnalités attendues pour chacun d'eux. La modélisation statique, mise à jour à travers un diagramme à **sept classes interconnectées**, sécurise la structure des données et formalise les relations exactes du domaine.

Implémentation

Dans ce chapitre, nous décrivons la phase d'implémentation de l'application dédiée à la gestion des actifs logistiques de l'établissement universitaire. L'objectif est de montrer comment les choix conceptuels et fonctionnels présentés précédemment ont été traduits en une application web opérationnelle.

L'application doit permettre à cinq types d'acteurs (administrateur, responsable logistique, magasinier, responsable de laboratoire et professeur) d'interagir avec un inventaire d'équipements, de suivre leur cycle de vie (acquisition, affectation, maintenance, réforme) et de formuler ou valider des demandes.

Nous présentons successivement :

- Les critères qui ont guidé nos choix technologiques,
- Les langages, frameworks et outils retenus,
- La structure de la base de données relationnelle,
- Enfin, un aperçu des principales fonctionnalités implémentées par acteur, accompagné d'extraits de code commentés.

L'ensemble de ce chapitre montre comment l'application répond concrètement aux besoins exprimés dans le cahier des charges.

4.1 Choix technologiques

4.1.1 Langages utilisés

HTML5

HTML5 est utilisé pour structurer l'ensemble des pages de l'application. Il définit la hiérarchie des éléments que l'utilisateur voit et manipule : formulaires d'enregistrement des équipements, tableaux de bord pour le responsable logistique, fiches détaillées du matériel, etc. Le choix de HTML5 s'impose naturellement car il s'agit du standard du web, compatible avec tous les navigateurs et facilement intégrable avec React (via JSX).

CSS3

La mise en forme visuelle est assurée par CSS3. Nous l'avons associé à Tailwind CSS pour gérer les couleurs, les espacements, la typographie et surtout l'adaptation aux différentes tailles d'écran (responsive). L'objectif était d'obtenir une interface claire et professionnelle, adaptée

aussi bien à un écran d'ordinateur qu'à un appareil mobile utilisé par le magasinier lors d'un inventaire terrain.

JavaScript

JavaScript constitue le cœur logique du projet. Il intervient à deux niveaux. Côté frontend, il gère les interactions utilisateur : soumission d'une demande d'équipement par un professeur, déclaration de panne, mise à jour dynamique de l'inventaire sans rechargement de page. Côté backend, avec Node.js et Express, il traite les requêtes, applique les règles de validation (par exemple, seul le responsable logistique peut valider une demande de réforme) et communique avec la base de données. L'intérêt d'utiliser JavaScript des deux côtés est d'avoir un langage unique, ce qui simplifie la transmission des données (au format JSON) et accélère le développement.

SQL et ORM (Prisma)

SQL reste le langage d'interrogation sous-jacent pour manipuler les données. Cependant, pour interagir de manière moderne et sécurisée avec notre base de données, nous avons intégré l'ORM **Prisma**. Prisma fait le pont entre le monde relationnel de PostgreSQL et notre backend en Node.js, en générant automatiquement un client TypeScript/JavaScript fortement typé. Cela sécurise les requêtes, élimine les risques d'injections SQL et accélère considérablement le développement des fonctionnalités de gestion d'inventaire.

4.1.2 Frameworks et bibliothèques

Pour structurer et accélérer le développement, nous avons retenu plusieurs frameworks et bibliothèques, chacun répondant à un besoin précis de l'application.

React.js pour l'interface utilisateur

React.js est la bibliothèque principale du frontend. Elle permet de découper l'interface en composants réutilisables. Chaque composant gère son propre état. Le choix de React se justifie par sa popularité, sa performance (virtual DOM) et sa compatibilité avec une architecture web mobile.

Vite pour l'outil de build

Vite remplace les outils traditionnels comme Webpack. Il assure un démarrage quasi instantané du serveur de développement et un rechargement à chaud (HMR). Cela a considérablement accéléré les phases de test et de mise au point des interfaces, notamment pour les formulaires d'enregistrement des mouvements d'équipement.

Tailwind CSS pour le style

Plutôt que d'écrire du CSS personnalisé, nous avons utilisé Tailwind CSS, une bibliothèque utilitaire. Elle fournit des classes. L'avantage est double : d'abord, nous n'avons pas à quitter le fichier composant pour styler un élément ; ensuite, le responsive se gère simplement.

React Router pour la navigation

React Router gère les routes de l'application. Il permet de passer d'une page à l'autre sans rechargement complet du navigateur. Il assure également la protection des routes

Node.js et Express pour le backend

Côté serveur, Node.js fournit l'environnement d'exécution JavaScript. Express, le framework qui repose sur Node.js, simplifie la création des API REST. Chaque fonctionnalité de l'application correspond à une route Express : GET /api/equipements pour consulter l'inventaire, POST /api/demandes pour soumettre une demande, PUT /api/pannes/:id pour mettre à jour le statut d'une panne, etc. Express gère aussi les middlewares, comme la vérification du token JWT avant d'accéder à une route protégée.

JWT pour l'authentification

Enfin, JWT (JSON Web Token) est le mécanisme d'authentification choisi. Lorsqu'un utilisateur se connecte avec son nom d'utilisateur et son mot de passe, le serveur lui renvoie un token signé. Ce token est ensuite inclus dans l'en-tête de chaque requête API. Le serveur vérifie sa validité et lit le rôle de l'utilisateur (admin, responsable logistique, magasinier, responsable laboratoire ou professeur) sans avoir à consulter la base de données à chaque fois. Cette approche sans état (stateless) est bien adaptée à une application web/mobile. Toutes les routes API protégées vérifient un token JWT contenant l'id et le rôle de l'utilisateur. Les composants React redirigent vers la page de connexion si le token est absent ou expiré.

4.1.3 Base de données

Le choix du système de gestion de base de données (SGBD) s'est porté sur une solution relationnelle robuste et performante : **PostgreSQL**. Ce choix est indispensable car les données de notre application sont fortement structurées et interconnectées : un équipement appartient à un laboratoire, un mouvement est lié à un utilisateur, et un signalement fait obligatoirement référence à un matériel précis.

PostgreSQL garantit le respect strict des contraintes d'intégrité (clés primaires et étrangères) et gère parfaitement les transactions complexes (propriétés ACID). Par exemple, lors de la réception d'une demande validée, l'écriture du nouveau matériel et la génération automatique du mouvement d'entrée s'effectuent au sein d'une même transaction indissociable. De plus, PostgreSQL gère nativement les types énumérés (Enums), ce qui sécurise les différents états des équipements, des demandes et des mouvements directement au niveau du serveur de données.

Base de données utilisée dans le projet

Pour le développement, nous avons utilisé MySQL Workbench comme environnement de gestion, avec une base locale. En production, l'application peut basculer sur PostgreSQL, qui offre des fonctionnalités avancées comme les types JSON et les contraintes de validation plus riches. Les deux SGBD étant relationnels et utilisant SQL standard, le passage de l'un à l'autre ne nécessite que peu de modifications.

La base contient six tables principales : utilisateurs, laboratoires, équipements, mouvements, demandes et pannes.

4.1.4 Outils de développement

Visual Studio Code

L'éditeur de code choisi est Visual Studio Code. Il est léger, gratuit et dispose d'un large écosystème d'extensions. Nous avons utilisé notamment ESLint pour la qualité du code JavaScript, Prettier pour le formatage automatique, Tailwind CSS IntelliSense pour l'autocomplétion des classes, et Thunder Client pour tester rapidement certaines API sans quitter l'éditeur. Son intégration native avec Git a facilité le suivi des modifications.

Git et GitHub

Le contrôle de version est assuré par Git, associé au dépôt distant GitHub. Chaque fonctionnalité (par exemple, l'ajout du formulaire de déclaration de panne) a été développée dans une branche séparée avant d'être fusionnée dans la branche principale après validation. GitHub a également servi à sauvegarder le code de manière sécurisée et à garder une trace de l'historique des modifications, ce qui est utile en cas de bug ou de retour en arrière nécessaire.

Postman

Postman a été utilisé pour tester les API indépendamment du frontend. Avant même d'écrire une seule ligne de code React, nous avons vérifié que chaque route Express (création d'un équipement, validation d'une demande, mise à jour d'une panne) retournait les bonnes données avec les bons codes HTTP. Cela a permis de détecter rapidement des erreurs de logique ou de permissions. Une fois l'API stabilisée, son intégration avec le frontend s'est faite sans surprise.

npm

npm est le gestionnaire de paquets par défaut pour l'écosystème JavaScript. Il a servi à installer toutes les dépendances du projet : React, Vite, Tailwind, React Router, Express, mysql2, jsonwebtoken, bcrypt, etc. Chaque membre de l'équipe a pu reproduire exactement le même environnement avec la commande `npm install`, à partir du fichier `package.json`. npm a également été utilisé pour lancer les scripts de développement (`npm run dev` pour Vite, `npm run server` pour Node).

Outil d'administration de la base de données (Prisma Studio)

Pour administrer, visualiser et manipuler facilement les données durant la phase de développement, nous utilisons **Prisma Studio**. Il s'agit d'une interface graphique moderne, légère et intégrée à notre ORM, lancée localement dans le navigateur. Contrairement à des outils traditionnels comme MySQL Workbench ou pgAdmin, Prisma Studio nous permet d'explorer, de filtrer et de modifier en temps réel les enregistrements de nos tables PostgreSQL tout en respectant scrupuleusement les relations et contraintes définies dans notre schéma de données.

4.2 Base de données

4.2.1 Approche technologique et Typage

L'application repose sur une base de données relationnelle **PostgreSQL**, modélisée et interrogée via l'ORM **Prisma**. Ce choix garantit une cohérence absolue à travers l'utilisation de types énumérés (*Enums*) qui verrouillent les valeurs possibles en base de données, limitant ainsi drastiquement les erreurs de saisie et les bugs logiques.

Les types énumérés définis dans notre schéma sont :

- **EtatEquipement** : NEUF, BON, MOYEN, PANNE, REFORME.
- **TypeDemande**: ACHAT, REMPLACEMENT, REFORME, EMPRUNT, LABORATOIRE.
- **StatutDemande** : EN_ATTENTE, VALIDEE, REFUSEE.
- **TypeMouvement** : ENTREE, SORTIE, TRANSFERT, MAINTENANCE.

4.2.2 Structure des tables

La base de données est structurée autour de **7 tables principales** interconnectées :

1. **Table Utilisateur** : Stocke les informations de connexion de l'ensemble du personnel (identifiant unique, nom complet, nom d'utilisateur, mot de passe hashé via bcrypt). Elle possède une relation directe vers la table Rôle.
2. **Table Rôle** : Table séparée dédiée à la gestion des privilèges. Elle liste les rôles stricts de l'application (Administrateur, Responsable Logistique, Magasinier, Responsable de Laboratoire, Professeur), permettant un contrôle d'accès granulaire (*RBAC*) sur l'ensemble des routes du système.
3. **Table Laboratoire** : Référence les espaces physiques (laboratoires et salles de l'établissement). Elle contient le nom, un code interne unique, la capacité maximale, et est liée à l'utilisateur qui en a la responsabilité.

4. **Table Equipement** : Table centrale du métier. Chaque actif y est enregistré avec son nom, sa référence constructeur unique, sa catégorie, sa quantité, son état (via l'Enum EtatEquipement), et sa localisation (clé étrangère vers la table Laboratoire).
5. **Table Demande** : Recense toutes les requêtes d'équipements formulées par les enseignants ou responsables. Elle stocke le type de demande (TypeDemande), la justification textuelle, la date, l'ID du demandeur, et son statut évolutif (StatutDemande).
6. **Table Mouvement** : Assure l'historique et la traçabilité physique complète du matériel. Elle enregistre le type de flux (TypeMouvement), la quantité, les laboratoires source et destination, l'utilisateur responsable de l'action, et l'horodatage précis.
7. **Table Signalement** : Enregistre les dysfonctionnements et incidents matériels déclarés sur le terrain. Elle associe l'équipement défectueux au déclarant, avec une description textuelle du problème et un statut de suivi (Signalé, En cours de réparation, Résolu).

4.2.3 Relations et Intégrité Référentielle

Grâce à Prisma et PostgreSQL, toutes les relations sont strictement maintenues :

Un **Utilisateur** est lié à un unique **Role** ; il peut émettre plusieurs **Demandes** et déclencher plusieurs **Mouvements**.

Un **Laboratoire** contient une collection d'**Equipements**.

Un **Equipement** peut faire l'objet de plusieurs **Signalements** au cours de sa vie et apparaît dans l'historique de la table **Mouvement**.

Toute suppression ou modification d'une entité clé déclenche des règles de gestion d'intégrité (blocage ou cascade) pour empêcher qu'un mouvement ou qu'un signalement ne devienne orphelin.

Ces relations permettent de naviguer facilement entre les données. Par exemple, pour connaître l'historique complet d'un équipement, on interroge la table mouvements avec son identifiant. Pour savoir qui a signalé une panne, on remonte à la table utilisateurs via le declarant_id. Cette structure relationnelle garantit à la fois la cohérence et la richesse des informations disponibles.

4.2.4 Cycle de vie et Flux Principal de Gestion

Pour automatiser la gestion logistique et limiter les interventions manuelles sources d'erreurs, l'application orchestre un flux numérique continu entre les acteurs :

Initiation : Le *Responsable de Laboratoire* identifie un besoin et formule une demande (ex: achat de nouveaux équipements) via son interface.

Validation : Le *Responsable Logistique* reçoit une alerte sur son tableau de bord, examine la justification et bascule le statut de la demande à VALIDEE.

Prise en charge physique : Dès réception du matériel, le *Magasinier* ouvre la liste des demandes validées dans son espace de travail pour enregistrer l'arrivée du matériel et éditer ses étiquettes d'identification.

Automatisation : Dès que le magasinier valide la réception en un clic, le système backend déclenche automatiquement deux actions simultanées en base de données : la création des fiches dans la table Equipement et l'insertion automatique d'une ligne de flux ENTREE dans la table Mouvement. Le stock et l'historique sont ainsi synchronisés instantanément sans double saisie.

4.3 Fonctionnalités implémentées

L'application propose une interface dynamique et responsive (adaptée aux ordinateurs ainsi qu'aux smartphones des magasiniers sur le terrain), où les menus s'adaptent strictement au rôle de l'utilisateur connecté.

1. Administrateur (Gestion système)

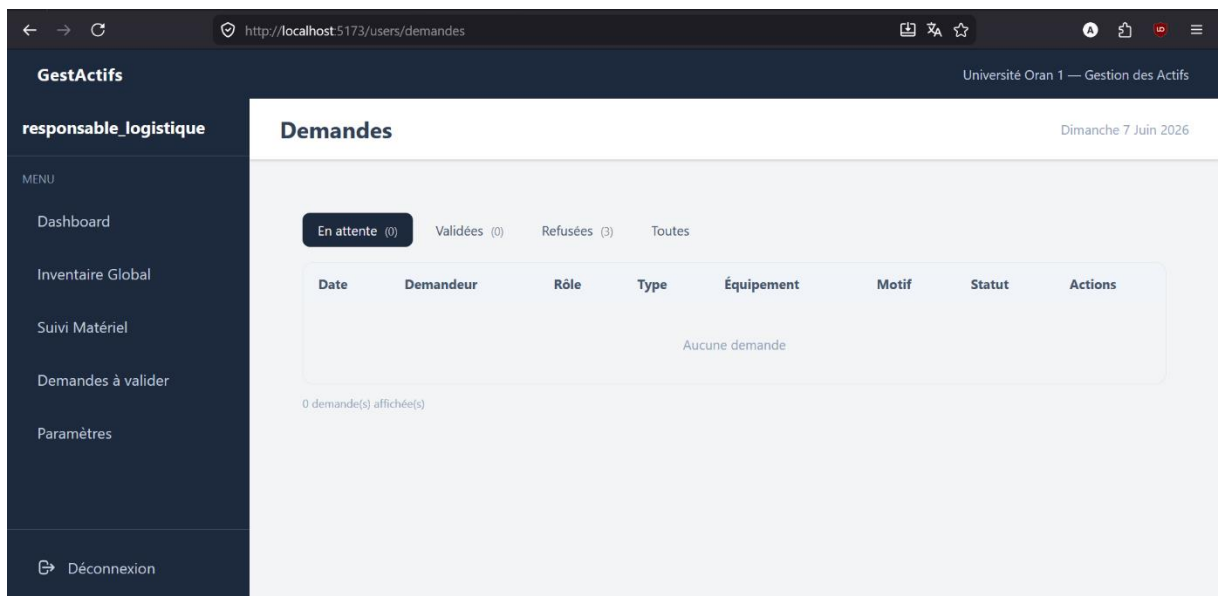
- **Gestion complète des utilisateurs** : Création, modification et suppression des comptes d'accès (liaison avec la table Role).
- **Réinitialisation de mot de passe** : Fonctionnalité de secours permettant à l'administrateur de réinitialiser le mot de passe d'un utilisateur ayant perdu ses accès.
- **Gestion des Laboratoires** : Configuration des espaces physiques et attribution de leur responsable.

The screenshot shows the 'Gestion Utilisateurs' page of the GestActifs application. The page features a sidebar with navigation options: Dashboard, Gestion Utilisateurs, Gestion Laboratoires, and Paramètres. The main content area displays a table of users with the following data:

Nom	Prénom	Email	Rôle	Actions
Meziane	Sara	labo@univ.dz	RESPONSABLE LABO	Modifier, Réinitialiser MDP, Supprimer
Benali	Louiza	admin@univ.dz	ADMIN	Modifier, Réinitialiser MDP, Supprimer
Berkaoui	Bilel	logistique@univ.dz	RESPONSABLE LOGISTIQUE	Modifier, Réinitialiser MDP, Supprimer
Bensaid	Lina	prof@univ.dz	PROFESSEUR	Modifier, Réinitialiser MDP, Supprimer
Hadj	Imene	magasinier@univ.dz	MAGASINIER	Modifier, Réinitialiser MDP, Supprimer

2. Responsable Logistique (Pilotage stratégique)

- **Dashboard analytique interactif** : Tableau de bord affichant des indicateurs et statistiques en temps réel sur l'état général du parc matériel, le volume des pannes et les demandes en attente.
- **Consultation globale de l'inventaire** : Filtrage multicritères avancé des équipements par laboratoire, état ou catégorie.
- **Traitement des demandes** : Espace centralisé pour valider ou rejeter les demandes d'achat, de remplacement, d'emprunt ou de réforme.
- **Gestion des Signalements** : Suivi et traitement des signalements de panne remontés par le terrain pour organiser les actions de maintenance.



3. Magasinier (Gestion opérationnelle et physique)

- **Réception depuis demandes validées** : Interface dédiée listant les ordres validés par la logistique, permettant d'enregistrer le matériel conforme en un clic.
- **Mouvement d'ENTREE automatique** : Génération transparente et automatisée de la traçabilité lors de la réception d'un équipement.
- **Impression d'étiquettes d'identification** : Génération et impression d'autocollants dotés d'identifiants uniques pour chaque équipement à coller physiquement sur le matériel.

- **Mouvements logistiques** : Saisie manuelle ou mise à jour des flux de TRANSFERT, MAINTENANCE ou SORTIE.
- **Traitement des demandes d'EMPRUNT** : Remise physique et enregistrement des sorties temporaires de matériel à destination des professeurs.

The screenshot shows a web application titled 'GestActifs' for 'Université Oran 1 — Gestion des Actifs'. The user is logged in as 'magasinier'. The main page is 'Réception Équipements', dated 'Dimanche 7 Juin 2026'. It features a table of equipment reception records with columns for Désignation, Référence, Catégorie, Qté, État, Laboratoire, and Action. A '+ Nouvel Équipement' button is in the top right of the table area.

Désignation	Référence	Catégorie	Qté	État	Laboratoire	Action
Projecteur Epson EB-X51	PROJ-EPSON-001	Audiovisuel	5	Neuf	Laboratoire Informatique	Imprimer
Switch Cisco Catalyst 2960	SWITCH-CISCO-001	Réseau	3	En panne	Laboratoire Réseaux	Imprimer
Projecteur Epson EB-X51		Audiovisuel	5	Neuf	—	Imprimer
PCjkznedjknze	PCJhzedihd	Achat	10	Neuf	Laboratoire Informatique	Imprimer
IMP	Imp-123	Achat	8	Neuf	Salle Tp 1	Imprimer
PPP	PPP	Achat	119	Bon état	Laboratoire Réseaux	Imprimer

4. Responsable de Laboratoire (Gestion locale)

- **Supervision de zone** : Consultation exclusive des équipements affectés à son entité, avec vérification en temps réel de leur **Disponibilité & État fonctionnel**.
- **Formulation de demandes de LABORATOIRE** : Création de requêtes structurées pour son entité (achat, remplacement, réforme d'équipements).
- **Validation intermédiaire** : Traitement et validation des demandes de matériel pédagogique ponctuel soumises par les professeurs pour leurs séances dans ses salles.

← → ↻ http://localhost:5173/users/disponibilite

responsable_labo

MENU

- Dashboard
- Mes Équipements
- Disponibilité & État
- Mes Demandes
- Demandes de Salle
- Paramètres
- 🔗 Déconnexion

Disponibilité & État

Dimanche 7 Juin 2026

DS	DS	Achat	1	Maraval	Neuf	—
memew	memew	Réforme	3	memew	Neuf	—
Imp-123	IMP	Achat	8	Salle tp1	Neuf	—
PPP	PPP	Achat	119	AA	Bon état	—
PCjhzedihd	PCjkznedjknze	Achat	10	pcpc	Neuf	—
PROJ-EPSON-001	Projecteur Epson EB-X51	Audiovisuel	5	Réserve	Neuf	—
SWITCH-CISCO-001	Switch Cisco Catalyst 2960	Réseau	3	Salle Réseau B2	En panne	—
P	P	P	2	TP1	Neuf	—
res	res	res	4	res1	Neuf	—
A	A	A	1	A	Neuf	—
tp	tp	rp	6	Salle tp1	Neuf	—

5. Professeur (Usage pédagogique)

- **Consultation locale** : Visualisation du matériel disponible dans la salle ou le laboratoire qui lui est attribué pour sa séance de cours ou de TP.
- **Demande d'emprunt d'équipement** : Formulaire permettant de solliciter du matériel mobile auprès du magasin central pour une durée déterminée.
- **Demande Laboratoire** : Requête adressée à son responsable pour l'amélioration ou l'aménagement de sa salle d'enseignement.
- **Signalement d'incidents** : Déclaration instantanée d'un dysfonctionnement sur un équipement (bouton "Déclarer un signalement" ouvrant un modal pour décrire la panne), alimentant directement la table Signalement.

GestActifs

professeur

MENU

Dashboard

Demander Équipement

Demander Salle

Signaler Équipement

Historique

Paramètres

Université Oran 1 — Gestion des Actifs

Dimanche 7 Juin 2026

Dashboard

Historique des Activités

9

Total Activités

3

Demandes Équipement

3

Demandes Salle

3

Signalements

Filtrer par type

Tout

Équipements

Salles

Signalements

Filtrer par date

jj/mm/aaaa

Type	Action	Détails	Référence	Date	Statut
Équipement	Demande équipement	Ordinateur Portable (Achat)	DEQ-001	15/04/2026	En attente

Conclusion et perspectives

Ce projet avait pour objectif de concevoir et développer une application web dédiée à la gestion des actifs logistiques au sein d'un établissement universitaire, en réponse aux limites des méthodes manuelles encore largement utilisées.

Au terme de ce travail, l'ensemble des objectifs fixés a été atteint. L'application permet à cinq acteurs distincts (administrateur, responsable logistique, magasinier, responsable de laboratoire et professeur) d'interagir avec un inventaire centralisé et accessible en temps réel. Chaque acteur dispose d'une interface personnalisée selon son rôle, ce qui garantit la sécurité des accès et la cohérence des données. Les fonctionnalités principales ont été implémentées avec succès : gestion des comptes, suivi automatisé des équipements, traçabilité des mouvements, gestion des emprunts, signalement des incidents sur le terrain, **ainsi qu'un système de notifications intégrées permettant d'alerter en temps réel les acteurs concernés**. Le pilotage global est assuré via un tableau de bord analytique interactif. L'authentification par JWT assure la protection de toutes les routes de l'application, et la base de données relationnelle PostgreSQL, couplée à l'ORM Prisma, garantit l'intégrité absolue des informations à travers ses sept tables interconnectées et ses types énumérés.

Cependant, comme tout projet réalisé dans un cadre académique, cette application présente certaines limites. Elle a été développée et testée dans un environnement local, sans déploiement réel sur un serveur de production. De plus, elle est actuellement configurée pour le département informatique, ce qui restreint son périmètre d'utilisation initial. Par ailleurs, des fonctionnalités avancées de gestion des alertes de maintenances préventives planifiées n'ont pas encore été intégrées.

Ces limites ouvrent naturellement la voie à plusieurs perspectives d'amélioration. La première et la plus importante serait d'étendre l'application à l'ensemble de l'établissement universitaire, en l'adaptant à tous les départements de la faculté, suivie d'un déploiement officiel sur un serveur cloud de production. Le développement d'une application mobile native ou PWA (Progressive Web App), complémentaire à la version web, faciliterait encore davantage le travail des magasiniers lors des inventaires physiques sur le terrain grâce à la numérisation directe des étiquettes de matériel. Enfin, l'intégration d'algorithmes prédictifs permettrait au responsable logistique d'anticiper le renouvellement des équipements obsolètes et d'optimiser le budget global de l'établissement.

En synthèse, ce projet constitue une contribution concrète à la digitalisation des services universitaires. Il démontre qu'il est possible, avec des technologies web modernes comme

l'écosystème React, Node.js et PostgreSQL, de répondre efficacement à des besoins réels de gestion, de traçabilité et d'organisation au sein d'une institution académique.

Références et Bibliographie

Auth0. (2024). *JSON Web Tokens – Introduction*. <https://jwt.io/introduction>

Banks, A., & Porcello, E. (2020). *Learning React: Modern patterns for developing React apps* (2e éd.). O'Reilly Media.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7e éd.). Pearson.

Evan You & Vite Team. (2024). *Vite – Next generation frontend tooling*. <https://vitejs.dev>

Flanagan, D. (2020). *JavaScript: The definitive guide* (7e éd.). O'Reilly Media.

GitHub, Inc. (2024). *GitHub documentation*. <https://docs.github.com>

Microsoft. (2024). *Visual Studio Code documentation*. <https://code.visualstudio.com/docs>

Mozilla Developer Network. (2024). *CSS – MDN Web Docs*.
<https://developer.mozilla.org/fr/docs/Web/CSS>

Mozilla Developer Network. (2024). *HTML5 – MDN Web Docs*.
<https://developer.mozilla.org/fr/docs/Web/HTML>

Mozilla Developer Network. (2024). *JavaScript – MDN Web Docs*.
<https://developer.mozilla.org/fr/docs/Web/JavaScript>

npm, Inc. (2024). *npm documentation*. <https://docs.npmjs.com>

OpenJS Foundation. (2024). *Node.js documentation*. <https://nodejs.org/en/docs>

Postman, Inc. (2024). *Postman learning center*. <https://learning.postman.com>

Prisma Team. (2026). *Prisma ORM Documentation - Next-generation Node.js and TypeScript ORM*. <https://www.prisma.io/docs>

React Team. (2024). *React – A JavaScript library for building user interfaces*. Meta Open Source. <https://react.dev>

React Training. (2024). *React Router documentation*. <https://reactrouter.com>

Strong, T. J. (2024). *Express.js – Fast, unopinionated, minimalist web framework for Node.js*.
<https://expressjs.com>

Tailwind Labs. (2024). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>

The PostgreSQL Global Development Group. (2024). *PostgreSQL documentation*.

<https://www.postgresql.org/docs>

Wexler, S. (2021). *Node.js in practice*. Manning Publications.